ANAGRAM CHECKER

Using Pointers and the Character-Frequency Counting Approach

Name	**:**	Rudra Pratap Singh
SAP ID	**:**	590032811
Batch	**:**	16

1. Problem Statement

Write a C program that reads two strings, s and t, both containing only lowercase characters. The strings must be passed to a function named check_anagram() using their addresses, and pointers must be used to pass the strings between the calling function and check_anagram().

The function check_anagram() must:

- Determine whether s and t contain the same characters with the same frequencies.
- Print "Anagram" if the two strings are valid anagrams, otherwise print "Not Anagram".
- Treat the order of characters as irrelevant.
- Treat strings of different lengths as not anagrams.
- Use a character-frequency-counting approach to solve the problem.

2. Approach

Two strings are anagrams when they are made of exactly the same letters, each occurring the same number of times, only in a different order. Sorting both strings and comparing them would work, but counting is simpler and faster, so a frequency array is used.

Since the input contains only lowercase letters, an integer array freq[26] is enough: index 0 stands for 'a', index 1 for 'b', and so on up to index 25 for 'z'. The index of any letter ch is found with the expression ch - 'a'.

3. Algorithm

1. Start. Read the strings s and t in main().
2. Call check_anagram(s, t). Only the addresses of the strings are passed, and they are received in the pointers char *s and char *t.
3. If strlen(s) is not equal to strlen(t), print "Not Anagram" and stop.
4. Create an array freq[26] and set every element to 0.
5. Traverse s with a pointer and increase freq[*p - 'a'] for each character.
6. Traverse t with a pointer and decrease freq[*p - 'a'] for each character.
7. Check all 26 elements of freq. If any element is not zero, print "Not Anagram" and stop.
8. If all elements are zero, print "Anagram".
9. Stop.

4. Source Code

```
#include <stdio.h>
#include <string.h>

#define MAX 1000
#define ALPHABETS 26

/* Checks whether the strings pointed to by s and t are anagrams.
   Uses pointers to access the characters and a frequency array
   to count how many times each letter occurs. */
void check_anagram(char *s, char *t)
{
    int freq[ALPHABETS] = {0};
    char *p;
    int i;

    /* Strings of different lengths can never be anagrams */
    if (strlen(s) != strlen(t))
    {
        printf("Not Anagram\n");
        return;
    }

    /* Increase the count for every character of s */
    for (p = s; *p != '\0'; p++)
        freq[*p - 'a']++;

    /* Decrease the count for every character of t */
    for (p = t; *p != '\0'; p++)
        freq[*p - 'a']--;

    /* If every count is back to zero, the frequencies are equal */
    for (i = 0; i < ALPHABETS; i++)
    {
        if (freq[i] != 0)
        {
            printf("Not Anagram\n");
            return;
        }
    }

    printf("Anagram\n");
}

int main()
{
    char s[MAX], t[MAX];

    printf("String s: ");
    scanf("%999s", s);

    printf("String t: ");
    scanf("%999s", t);

    printf("Output: ");
    check_anagram(s, t);   /* addresses of the strings are passed */

    return 0;
}
```

5. Explanation of the Code

- #define MAX 1000 and #define ALPHABETS 26 set the maximum string size and the number of lowercase letters, which avoids "magic numbers" in the program.

- main() reads both strings with scanf("%999s", ...). The width limit 999 prevents the input from overflowing the 1000-character array.
- The call check_anagram(s, t) passes the base addresses of the two character arrays, so the function works directly on the original strings through the pointers char *s and char *t.
- The length check at the start handles strings of unequal length immediately, since they can never be anagrams.
- The pointer p moves from the start of each string up to the null character '\0'. The expression *p gives the current character, and *p - 'a' converts it to an index from 0 to 25.
- Counting up for s and down for t means that matching letters cancel out. If the strings are anagrams, every element of freq returns to zero.

6. Test Cases and Output

The program was compiled with gcc -Wall -Wextra (no warnings) and tested with the four given examples plus one extra case of unequal length.

Summary of results

No.	String s	String t	Output	Reason
1	listen	silent	Anagram	Same letters, same counts
2	hello	world	Not Anagram	Different letters
3	anagram	nagaram	Anagram	Same letters, same counts
4	rat	car	Not Anagram	Different letters (t vs c)
5	ab	abc	Not Anagram	Different lengths

Program runs (as seen in the terminal)

Example 1

```
String s: listen
String t: silent
Output: Anagram
```

Example 2

```
String s: hello
String t: world
Output: Not Anagram
```

Example 3

```
String s: anagram
String t: nagaram
Output: Anagram
```

Example 4

```
String s: rat
String t: car
Output: Not Anagram
```

7. Conclusion

The program correctly identifies whether two lowercase strings are anagrams. It passes the strings to check_anagram() by address, accesses them through pointers, and uses a frequency array to compare the characters. The method runs in O(n) time, where n is the length of the strings, and uses a fixed amount of extra memory (26 integers), which makes it more efficient than sorting both strings.